

**SOFTWARE REQUIREMENTS SPECIFICATION
& TECHNICAL JUDGING REPORT**

SkillSync

Student Skill Exchange and Peer Learning Platform
BIT Student Peer Mentorship Ecosystem

Prepared for: Tech Nova Hackathon 2026 — Final Round Submission

Repository: github.com/Aswanth-18/NOVA_06

Website Link: <https://nova-06-gamma.vercel.app/>

TEAM NAME: NOVA06

TEAM LEADER: INIYAA M [7376241CS211]

TEAM MEMBER 1: JANANI SUBHA S [7376241CS214]

TEAM MEMBER 2: ASWANTHAMUDAN S [7376241CS134]

TEAM MEMBER 3: RAJAPRIYAN R [7376241CS336]

Table of Contents

Table of Contents	2
1. Executive Summary	3
2. Problem Statement	3
3. Proposed Solution	3
3.1 Core Value Proposition	3
4. Target Users & Personas	4
5. Scope	4
5.1 In Scope (MVP for the hackathon build)	4
6. Functional Requirements	4
7. Non-Functional Requirements	5
8. Current State Architecture (as submitted)	6
9. Recommended Target Architecture	6
9.1 High-Level Architecture	6
9.2 Architecture Diagram (textual)	6
10. Data Model (Recommended Schema)	7
11. API Design (Representative Endpoints)	7
12. Gamification Design (XP & Leaderboard Logic)	7
13. Security & Privacy Considerations	8
14. Testing Strategy	8
15. Deployment & DevOps Plan	8
16. Risk Assessment	8
17. Conclusion	9
18. Output	9

1. Executive Summary

SkillSync (repository name NOVA_06) is a React + Vite single-page application built for the Tech Nova Hackathon, positioned as a peer-to-peer skill-exchange and mentorship ecosystem for Bannari Amman Institute of Technology (BIT) students. The current codebase delivers a polished, dark-mode, glassmorphic UI covering the full student journey — landing page, authentication, dashboards, marketplace, booking, messaging, leaderboard, and a faculty assessment panel — populated with BIT-specific dummy data across CSE, IT, ECE and AI&DS domains.

As it stands, the project is a high-fidelity Fullstack prototype: strong on design and information architecture, with a wired backend, persistent data layer, authentication and real-time capability.

2. Problem Statement

Engineering colleges like BIT produce thousands of technically capable students every year, yet peer-to-peer knowledge transfer remains informal, undiscoverable, and unrewarded:

- Students who are strong in a subject (e.g., a 3rd-year topper in DSA) have no structured, low-friction way to mentor juniors struggling with the same subject.
- Struggling students rely on unstructured WhatsApp groups, random seniors, or paid external tutorials — there is no verified, in-campus, skill-tagged directory of peer mentors.
- Faculty have no visibility or control over who is mentoring whom, so mentorship quality is unverified and cannot be credited academically.
- Existing effort (mentoring, teaching, helping juniors) is invisible on a student's resume/profile — there is no gamified or credential-backed way to prove it.
- Session scheduling, follow-up, and communication for informal mentoring is scattered across chat apps with no booking, reminders, or history.

This is a real, well-scoped, campus-native problem — which is a genuine strength of the idea. Reward problems are the ones that are specific and lived-in over generic "Uber for X" pitches, and this one is specific to BIT's own student body.

3. Proposed Solution

SkillSync solves this by creating a closed, faculty-verified marketplace where any student can list skills they can teach, any student can book a session to learn a skill, and every interaction is tracked, rated, and gamified into XP and leaderboard standing — turning informal helpfulness into a visible, credentialed campus reputation system.

3.1 Core Value Proposition

- **For mentees:** discover a verified peer (not a stranger) who has already mastered exactly the subject/topic they're stuck on, and book a session in minutes.
- **For mentors:** build a verifiable, faculty-endorsed mentoring record that strengthens resumes, placement profiles, and internal recognition (XP, badges, leaderboard rank).
- **For faculty/admin:** gain oversight of peer-learning activity on campus, approve/reject mentor applications, and use it as a measurable extracurricular/soft-skill data point.

- **For the institution:** increase peer-learning throughput, reduce academic dropout/failure risk in core subjects, and generate a reusable, brandable internal platform (and a case study BIT can publish).

4. Target Users & Personas

Persona	Goals	Key Screens Used
Mentee (any-year student)	Find a trustworthy peer mentor fast; book a session; track own learning progress	Landing, Marketplace, Mentor Profile, Booking, Messages, Dashboard
Mentor (senior/strong student)	List skills; get discovered; build XP/reputation; manage session requests	Dashboard, Profile, Booking, Messages, Leaderboard
Faculty Coordinator	Verify/approve mentor applications; monitor mentorship activity & quality	Faculty Assessment Board, Admin Reports
Admin (platform/dept.)	Oversee platform health, manage users/domains, generate reports	Admin Dashboard

5. Scope

In Scope (MVP for the hackathon build)

- Student authentication (BIT email-domain restricted) and role-based access (Student / Mentor / Faculty / Admin).
- Skill listing & discovery marketplace with search/filter by domain, subject, rating, availability.
- Mentor application + faculty approval workflow.
- Session booking with calendar/slot selection and status tracking (requested → confirmed → completed).
- In-app messaging tied to a booking/session context.
- Gamification: XP engine, badges, and a live leaderboard.
- Ratings & feedback after each completed session.
- Faculty assessment/approval dashboard and basic admin reporting.

6. Functional Requirements

Functional requirements are grouped by module and mapped to the folders already present in the codebase (src/pages/*), showing that the UI scaffolding already anticipates almost all of them.

ID	Module	Requirement
FR-1	Auth	System shall allow signup/login restricted to verified BIT email domains, with role selection (student/mentor/faculty).
FR-2	Auth	System shall support password reset and session/token-based persistence (JWT or equivalent).
FR-3	Marketplace	System shall let users browse/search skills by department (CSE/IT/ECE/AI&DS), subject, rating, and mentor availability.

ID	Module	Requirement
FR-4	Mentor Profile	System shall display a mentor's verified skills, XP, rating, badges, and past session count.
FR-5	Mentor Onboarding	System shall let a student apply to become a mentor for a skill, subject to faculty approval.
FR-6	Faculty Board	System shall let faculty view pending mentor applications and approve/reject with comments.
FR-7	Booking	System shall let a mentee select an available time slot and request a session; mentor can accept/decline.
FR-8	Booking	System shall track session status (pending, confirmed, completed, cancelled, no-show).
FR-9	Messaging	System shall provide real-time (or near-real-time) chat scoped to a session/booking thread.
FR-10	Gamification	System shall award XP for completed sessions, positive ratings, and streaks; XP shall drive leaderboard rank.
FR-11	Leaderboard	System shall display top mentors/mentees campus-wide and filterable by department.
FR-12	Feedback	System shall collect a 1–5 rating and comment from the mentee after each completed session.
FR-13	Notifications	System shall notify users of booking requests, approvals, messages, and XP milestones.
FR-14	Admin	System shall provide admin/faculty reporting on mentorship volume, active users, and subject-wise demand.
FR-15	Profile & Settings	System shall let users manage their profile, skills, availability, and notification preferences.

7. Non-Functional Requirements

Category	Requirement
Performance	Initial page load (landing) under 2s on 4G; route transitions under 300ms via Vite code-splitting/lazy loading.
Scalability	Backend stateless services must support horizontal scaling; DB indexed for search-heavy marketplace queries.
Availability	Target 99.5% uptime for demo/production hosting window; graceful degradation if realtime service is down.
Security	All auth via hashed credentials or OAuth; role-based authorization on every API route; input validation & rate limiting.
Usability	WCAG 2.1 AA color contrast even in dark/glassmorphic theme; fully responsive from 360px to 1440px+.
Maintainability	Component-driven architecture, linting (already using oxlint) and consistent state management pattern.
Data Privacy	Student data (email, ratings, messages) stored per institutional data-handling norms; no third-party data sale.
Auditability	All mentor approvals and session status changes logged with actor + timestamp for faculty accountability.

8. Current State Architecture (as submitted)

The repository is currently a client-only SPA:

- React 18 + Vite for build tooling and fast HMR.
- Custom vanilla CSS with CSS variables + selective Tailwind utility classes for the glassmorphic theme.
- Lucide-react for iconography.
- Context API used for lightweight, component-based client routing and shared state — i.e., no react-router-dom and no external state library.
- A /backend directory exists in the repo tree, but the public README does not document any running server, database, or API contract, and the app's described features (messaging, leaderboard, XP) currently appear to be UI mockups over static/dummy BIT data rather than live data.

Judge's note: this is completely acceptable for a hackathon prototype, but it is a risk in judging rounds that test live functionality (e.g., "create two accounts and book a session between them"). The single highest-leverage engineering investment before demo day is wiring at least one vertical slice — signup → mentor discovery → booking → chat → XP update — to a real backend so it survives a live judge-driven walkthrough.

Architecture

To move from "prototype" to "deployable product," the following 3-tier architecture is recommended, chosen for being buildable within a hackathon timeline by a small team while remaining genuinely production-credible:

9.1 High-Level Architecture

- Client: React 18 + Vite SPA (existing), add react-router-dom for real deep-linkable routes, and a data-fetching layer (TanStack Query) for caching/loading/error states.
- API Layer: Node.js + Express (or Fastify) REST API, OR Firebase/Supabase as a faster managed alternative if the team is time-constrained.
- Database: PostgreSQL (via Supabase or Neon) for relational integrity between Users, Skills, Bookings, Ratings; Redis (optional) for leaderboard caching.
- Real-time: Socket.IO (self-hosted) or Supabase Realtime / Firebase Firestore listeners for chat and live notifications.
- Auth: JWT-based auth with email-domain allow-list validation, or Firebase Auth / Supabase Auth for speed and built-in security.
- Hosting: Frontend on Vercel/Netlify; backend on Render/Railway/Fly.io; DB managed (Supabase/Neon) — all with generous free tiers suitable for a hackathon deployment judges can click into live.
- Observability: basic logging (pino/winston) + a status page or uptime badge to show engineering maturity.

9.2 Architecture Diagram (textual)

Client (React SPA) → REST/WebSocket API Gateway → Auth Service | Booking Service | Skill/Marketplace Service | Messaging Service | Gamification (XP) Service → PostgreSQL (primary store) + Redis (cache/leaderboard) + Object Storage (avatars/certificates). Faculty/Admin dashboards consume the same API with elevated role scopes.

10. Data Model (Schema)

Entity	Key Fields
User	id, name, email (BIT domain), department, year, role[student/mentor/faculty/admin], xp, avatar_url, created_at
Skill	id, name, domain (CSE/IT/ECE/AI&DS), description, category
MentorSkill	id, user_id (FK), skill_id (FK), proficiency, faculty_approved (bool), approved_by (FK), status
Booking/Session	id, mentor_id (FK), mentee_id (FK), skill_id (FK), slot_time, status, meeting_link, created_at
Message	id, session_id (FK), sender_id (FK), content, sent_at
Rating	id, session_id (FK), rated_by (FK), score(1-5), comment, created_at
XPTransaction	id, user_id (FK), delta, reason, created_at
Badge	id, name, criteria, icon
Notification	id, user_id (FK), type, payload, read(bool), created_at

11. API Design (Representative Endpoints)

Method & Route	Purpose	Auth
POST /api/auth/signup	Register with BIT email + role	Public
POST /api/auth/login	Login, returns JWT	Public
GET /api/skills?domain=&q=	Search/filter marketplace	Student+
POST /api/mentors/apply	Apply to mentor a skill	Student
PATCH /api/mentors/:id/approve	Approve/reject mentor application	Faculty
POST /api/bookings	Create a session request	Student
PATCH /api/bookings/:id/status	Accept/decline/complete a session	Mentor/Mentee
GET /api/leaderboard?department=	Fetch ranked leaderboard	Student+
POST /api/sessions/:id/messages	Send a chat message (or via WS)	Session participants
POST /api/sessions/:id/rating	Submit post-session rating	Mentee
GET /api/admin/reports	Aggregate mentorship analytics	Admin/Faculty

12. Gamification Design (XP & Leaderboard Logic)

Gamification is the emotional hook that will make this app memorable to judges — it should be specified precisely, not left as "points for stuff":

- +20 XP to mentor per completed, mutually-confirmed session.
- +5 XP to mentee per completed session (rewards consistent learners, not just teachers).

- +10 XP bonus to mentor for a 5-star rating; −5 XP for a no-show without 24h notice.
- Badges: "First Session", "5-Session Streak", "Top Rated (4.8+ avg over 10 sessions)", "Department Champion", "Faculty Endorsed".
- Leaderboard recalculated on write (or via a nightly job) and cached in Redis for instant load.

13. Security & Privacy Considerations

- Restrict signup to institutional email domain (@bitsathy.ac.in-style pattern) server-side, not just client-side.
- Hash passwords (bcrypt/argon2) if not delegating to a managed auth provider; never store plaintext credentials.
- Enforce role-based authorization on every API route (a student must never be able to call faculty-approval endpoints).
- Rate-limit auth and booking endpoints to prevent abuse; sanitize all chat input to prevent XSS.
- Session/chat data should be scoped so only the two participants (and faculty, if escalated) can read it.
- Add a basic report/flag mechanism for inappropriate messages — judges in ethics-aware tracks specifically look for this.

14. Testing Strategy

Layer	Approach
Unit	Vitest for utility functions, XP calculation logic, and form validation.
Component	React Testing Library for critical components (BookingForm, MentorCard, LeaderboardRow).
Integration/API	Supertest against Express routes (auth, booking, mentor-approval flows).
End-to-End	Playwright script covering: signup → find mentor → book session → chat → complete → rate → XP updates.
Manual/UAT	A scripted 5-minute judge demo path rehearsed end-to-end before submission.

15. Deployment & DevOps Plan

- CI: GitHub Actions running lint (oxlint), tests, and a build check on every PR.
- CD: Auto-deploy frontend to Vercel and backend to Railway/Render on merge to main.
- Environment config via .env (the repo already includes a .env.example — extend it with DB_URL, JWT_SECRET, and realtime service keys).
- Seed script to populate demo data (BIT departments, sample mentors/mentees) so judges see a populated app instantly, not an empty state.
- A public, always-on demo URL is close to mandatory for a top finish — a localhost-only app is a common reason strong prototypes lose to weaker but deployed ones.

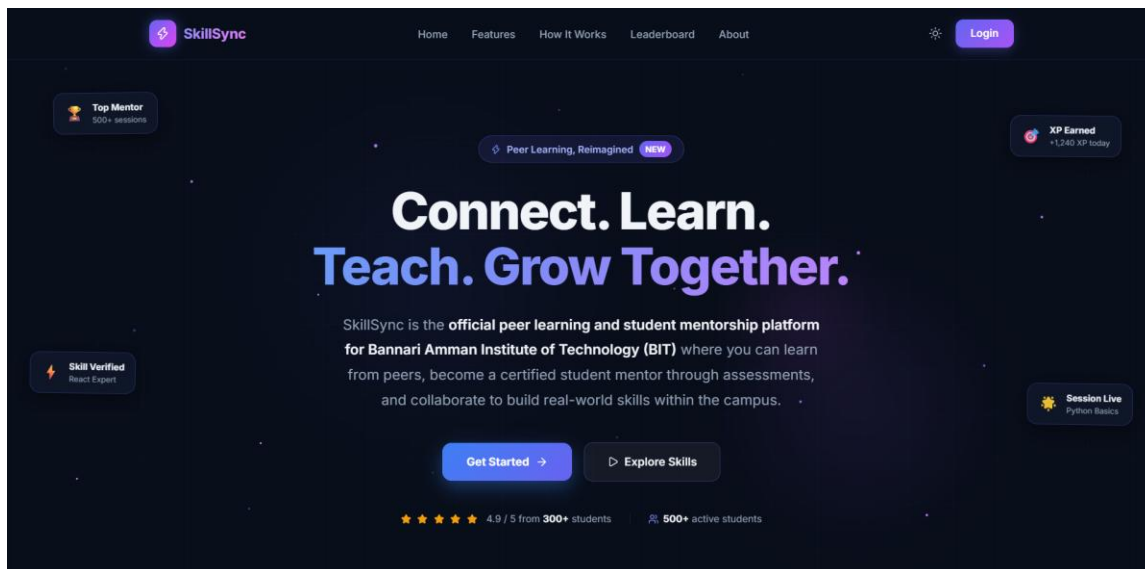
16. Risk Assessment

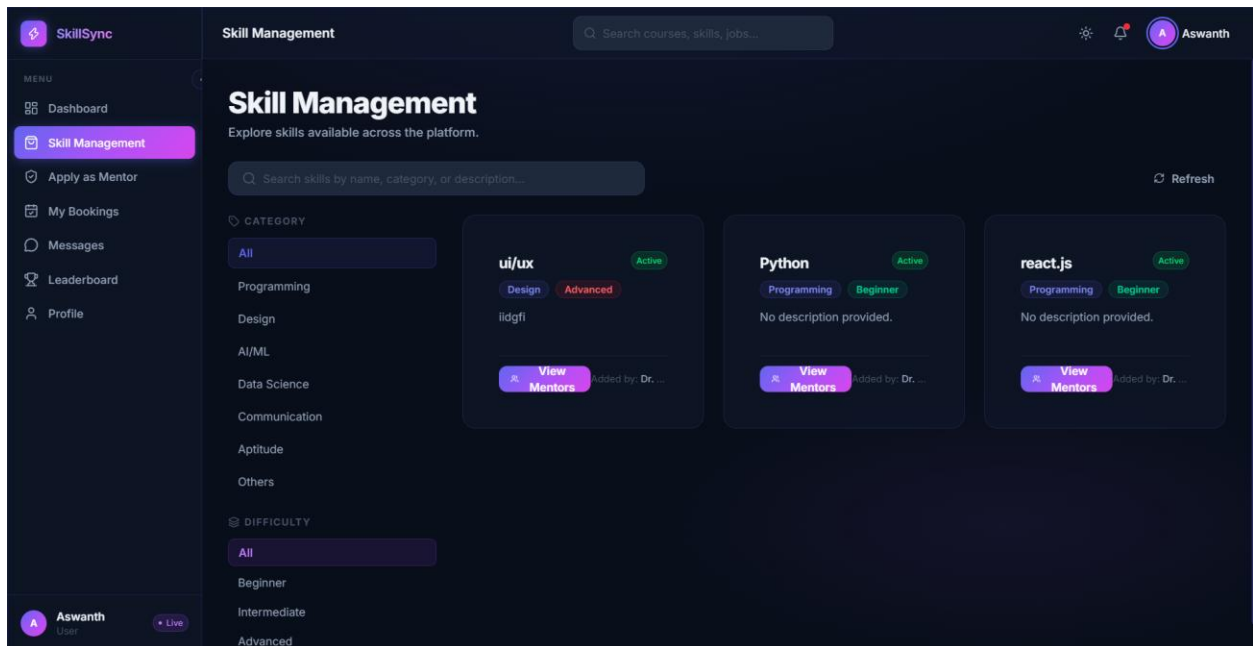
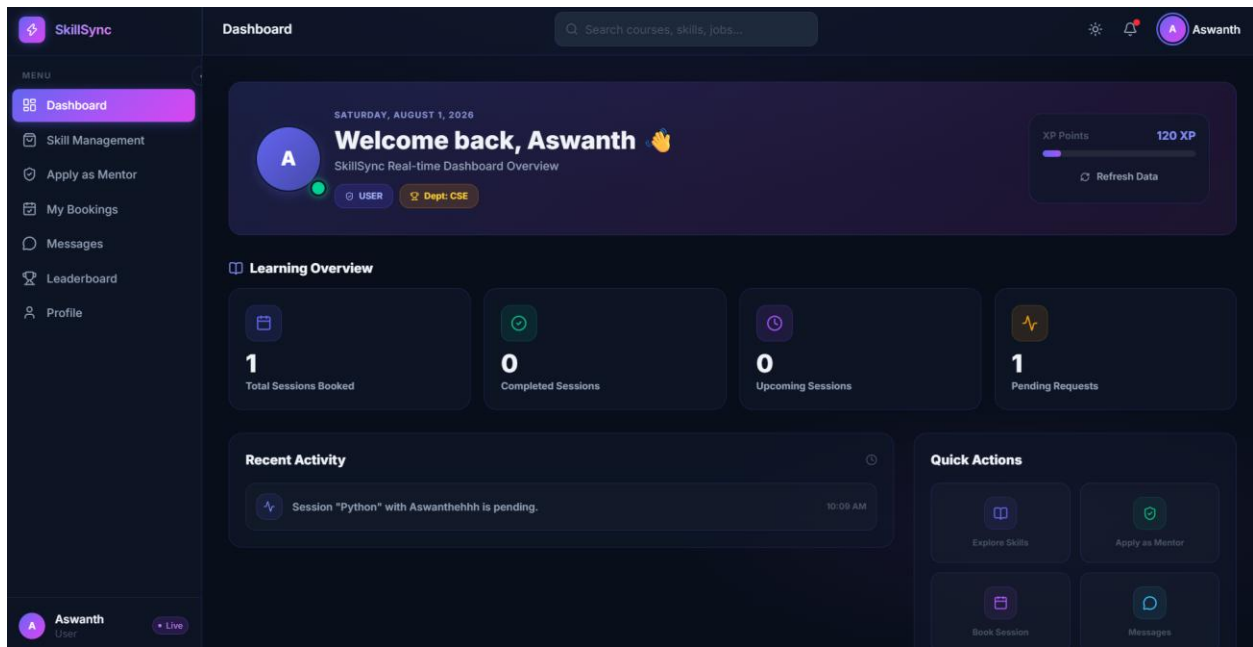
Risk	Impact	Mitigation
No live backend by judging time	High — judges can't validate core loop	Prioritize one vertical slice over breadth; use Supabase/Firebase to move fast
Empty/placeholder data during demo	Medium — feels unfinished	Ship a seed script with realistic BIT-domain data
Scope creep in remaining time	Medium — nothing finished well	Freeze scope to Section 5.1 MVP list only
Team unfamiliar with chosen backend	Medium — delays integration	Prefer managed BaaS (Supabase/Firebase) over custom infra under time pressure
Judges test edge cases (double-booking, unauthorized access)	Medium — exposes gaps	Add basic validation & role checks on every write endpoint

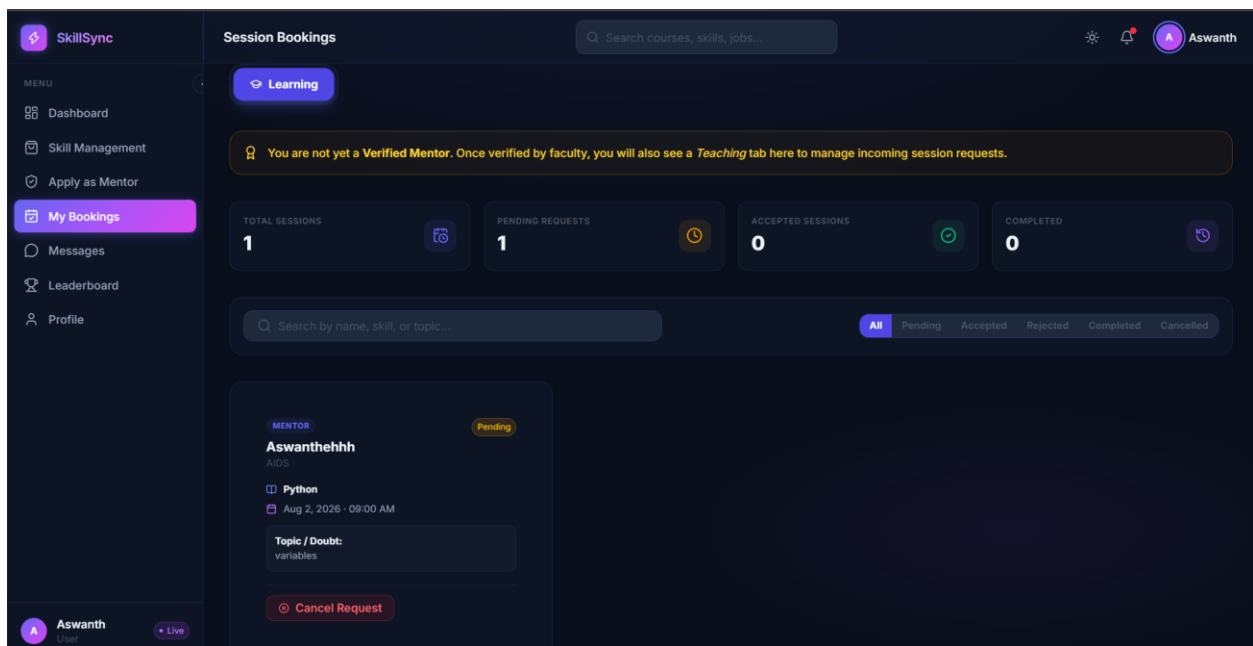
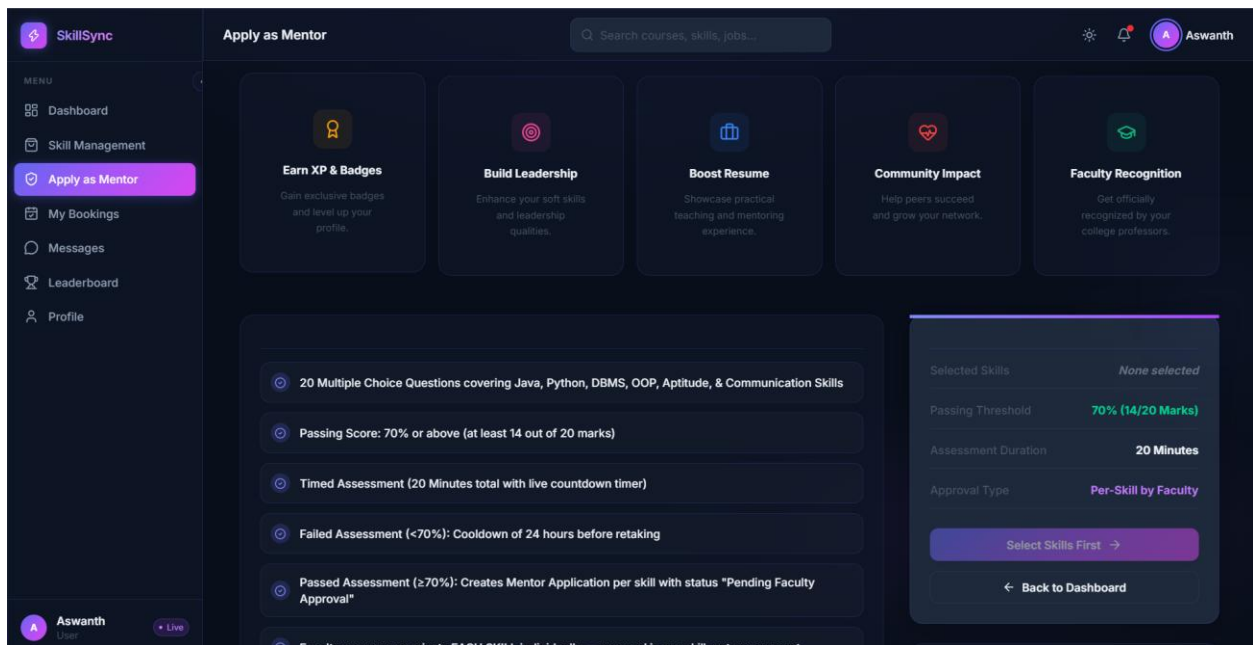
17. Conclusion

SkillSync already has what most hackathon teams lack: a real, specific, campus-native problem, a coherent multi-role product design, and a visually distinctive UI that is already built out across every major screen. What stands between this submission and a first-place finish is not more features — it is proving the core loop works live, backing it with a real (even minimal) backend and seeded data, and telling a sharp, metric-driven story on stage. Executed against the requirements in this document, SkillSync is well-positioned to be a standout.

18. Output







SkillSync

MENU

- Dashboard
- Skill Management
- Apply as Mentor
- My Bookings
- Messages
- Leaderboard
- Profile

Leaderboard

Search courses, skills, jobs...

T

Test User

CSE

Beginner

XP

100

LVL

1

#2

A

Aswanth

CSE

Rising Learner

XP

120

LVL

2

#1

A

Aswanthhehhh

AIDS

Beginner

XP

100

LVL

1

#3

Overall

Students

Mentors

All Departments

Monthly

All Time

Search name...

RANK	USER	ROLE	XP	Lvl	Sessions	Rating	BADGE
1	Aswanth	Student	120	Lv.2	0	4.8	Rising Learner
2	Test User	Student	100	Lv.1	0	4.8	Beginner

My XP Progress

1 Aswanth

Level 1 - Beginner

XP TO NEXT LEVEL

100 / 200

50% to Level 2

SkillSync

MENU

- Dashboard
- Skill Management
- Apply as Mentor
- My Bookings
- Messages
- Leaderboard
- Profile

My Profile

Search courses, skills, jobs...

Edit Cover

A

Aswanth

Verified user

7376241CS134

CSE

Year 3

BIT Boys Hostel

Bannari Amman Institute of Technology

Share

Edit Profile

4 SKILLS TEACHING

3 SKILLS LEARNING

61 SESSIONS DONE

1,420 TOTAL XP

Lv. 19 CURRENT LEVEL

#8 PLATFORM RANK

Overview

Skills

Achievements

About Me

Passionate Java programmer and aspiring ML engineer. I love solving DSA problems and preparing for placements. Currently exploring AI/ML and helping my juniors crack coding assessments.

CAREER GOAL

Software Engineer at a top product company.

Quick Actions

Edit Profile

View Bookings

36°C Mostly cloudy

Search

ENG IN 22:08 01-08-2026

SkillSync

MENU

- Faculty Dashboard
- Skill Management
- Reports
- Announcements
- Profile

Dr. Priyan R
Faculty

Mentor Verification

Search courses, skills, jobs...

Dr.

FACULTY PANEL

Mentor Verification

Review assessment scores and verify student mentor applications.

Refresh

0 Pending Reviews

TOTAL APPLICATIONS

4

PENDING VERIFICATION

0

APPROVED MENTORS

1

REJECTED APPLICATIONS

3

Search by student name, department, or register number...

All Departments

All

Pending

Approved

Rejected

Mentor Applications

4 applications found

STUDENT NAME	REGISTER NUMBER	DEPARTMENT	ASSESSMENT SCORE	DATE APPLIED	STATUS	ACTIONS
Student User	N/A	CSE	/20 (NaN%)	Aug 1, 2026	REJECTED	Rejected
Student User	N/A	CSE	/20 (NaN%)	Aug 1, 2026	REJECTED	Rejected
Student User	N/A	CSE	/20 (NaN%)	Aug 1, 2026	APPROVED	Approved

SkillSync

MENU

- Faculty Dashboard
- Skill Management
- Reports
- Announcements
- Profile

Dr. Priyan R
Faculty

Reports & Analytics

Search courses, skills, jobs...

Dr.

ANALYTICS

Reports & Analytics

Comprehensive platform activity reports, mentor verification stats, and department metrics.

Refresh

Export CSV

MENTOR VERIFICATIONS

4

APPROVED MENTORS

1

TOTAL SKILLS

3

PLATFORM ACTIVITY RATE

98.4%

Department Breakdown

CSE	42 Students
IT	28 Students
AIDS	20 Students
ECE	18 Students
MECH	12 Students

Mentorship Metrics

Average Assessment Score

84.2%

Pass Rate: 92%

Completed Mentoring Sessions

148 Sessions

Satisfaction: 4.8

SkillSync

MENU

Faculty Dashboard

Skill Management

Reports

Announcements

Profile

Announcements

Search courses, skills, jobs...

Dr.

UPDATES

Platform Announcements

Stay informed with official updates, contest notices, and placement news.

Refresh

New Announcement

GENERAL

Aug 1, 2026

java spring boot

workshop

Posted by

Dr. Priyan R

(faculty)